ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА

ЗВІТ
ДО ЛАБОРАТОРНОЇ РОБОТИ №7

«Робота з базами даних у Python»

з курсу «Професійний Python»
Варіант №3 — «Інтернет-магазин»

Виконав(ла): Вересюк Ірина

Група: ФЕП-32

Перевірив(ла):Куньо І.М.

Дата: 28.10.26

Львів — 2026

1. Мета роботи

Ознайомитися з основними принципами побудови шару зберігання даних (persistence layer) у Python-застосунках. Навчитися працювати з реляційними базами даних під керуванням СУБД SQLite, мовою SQL, специфікацією Python DB-API, параметризованими запитами, CRUD-операціями, транзакціями та принципами ACID. Опанувати патерн Repository, створення ORM-моделей SQLAlchemy, налаштування зв'язків між таблицями (relationships), виконання міграцій структури БД за допомогою Alembic та створення automated integration tests із використанням ізольованої тестової бази даних.

2. Індивідуальний варіант і постановка задачі

Варіант №3 — «Інтернет-магазин». Продовжити розвиток проєкту попередніх лабораторних робіт зі збереженням предметної області. Для товару зберігати назву, категорію, ціну та кількість. Вимоги варіанта:

1. Створити реляційні таблиці categories та products (зв'язок one-to-many).
2. Реалізувати класи доступу до даних CategoryRepository та ProductRepository на основі Repository Pattern.
3. Зберегти аналітичні функції попередньої роботи: додавання товару, обчислення сумарної вартості залишків, пошук товару за назвою, фільтрація за категорією та визначення найдорожчого товару.
4. Додати повний CRUD, зміну ціни та кількості товару.
5. Через стандартний Python DB-API реалізувати параметризований SQL-запит для фільтрації товарів за назвою категорії.
6. Реалізувати транзакційну операцію purchase_product(): атомарно перевірити залишок, зменшити кількість товару та зафіксувати операцію (commit), а в разі нестачі залишків або збою виконати відкат (rollback).
7. Створити та застосувати міграцію Alembic для додавання поля description до таблиці products.
8. Покрити persistence layer інтеграційними тестами (pytest) на ізольованій in-memory базі даних.

3. Структура проєкту

Проєкт організовано за стандартом src-layout з окремими каталогами для міграцій, тестів та звіту:

online_store/

├── pyproject.toml

├── alembic.ini

├── report/

│ └── Вересюк_Ірина_Лаб07_V3.pdf

├── migrations/

```
│   ├── env.py
│   └── versions/
│       └── 0001_add_description_to_products.py
├── src/
│   └── online_store/
│       ├── __init__.py
│       ├── database.py
│       ├── models.py
│       ├── repositories.py
│       ├── services.py
│       └── main.py
└── tests/
    ├── conftest.py
    └── test_repositories.py
```

4. Опис створених модулів

4.1. database.py

Модуль конфігурує фабрику сесій SQLAlchemy ORM та повертає низькорівневе підключення sqlite3 DB-API.

```
import sqlite3

from sqlalchemy import create_engine

from sqlalchemy.orm import sessionmaker



DATABASE_FILE = "store.db"

DATABASE_URL = f"sqlite:///{DATABASE_FILE}"



# Підключення для SQLAlchemy

engine = create_engine(
```

```
    DATABASE_URL,
    echo=False,
)

SessionLocal = sessionmaker(
    bind=engine,
    autoflush=False,
    expire_on_commit=False,
)

def get_db_connection(db_path: str = DATABASE_FILE) -> sqlite3.Connection:
    """Підключення через стандартний модуль sqlite3 (DB-API)."""
    connection = sqlite3.connect(db_path)
    connection.row_factory = sqlite3.Row
    return connection
```

Створює екземпляр Engine для SQLite та фабрику сесій SessionLocal без автокоміту для ручного контролю транзакцій. Функція get_db_connection() реалізує з'єднання через стандартний модуль sqlite3 для виконання прямих SQL-запитів через Python DB-API.

4.2. models.py

Описує схему реляційної бази даних засобами декларативного стилю SQLAlchemy 2.0.

```
from __future__ import annotations
from sqlalchemy import Float, ForeignKey, Integer, String
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column, relationship

class Base(DeclarativeBase):
    pass

class Category(Base):
```

```
    __tablename__ = "categories"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    name: Mapped[str] = mapped_column(String(100), unique=True, nullable=False)

    products: Mapped[list["Product"]] = relationship(
        back_populates="category",
        cascade="all, delete-orphan",
    )

class Product(Base):
    __tablename__ = "products"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    name: Mapped[str] = mapped_column(String(150), nullable=False)
    price: Mapped[float] = mapped_column(Float, nullable=False)
    quantity: Mapped[int] = mapped_column(Integer, nullable=False, default=0)
    category_id: Mapped[int] = mapped_column(ForeignKey("categories.id"), nullable=False)

    # Поле description додається міграцією за умовами варіанта 3
    description: Mapped[str | None] = mapped_column(String(255), nullable=True)

    category: Mapped["Category"] = relationship(back_populates="products")

    @property
    def total_value(self) -> float:
        """Вартість залишку для даного товару."""
```

```
        return round(self.price * self.quantity, 2)
```

Декларує таблиці categories та products. Визначено первинні ключі (id), зовнішній ключ category_id для зв'язку One-to-Many (relationship), каскадне видалення, обмеження unique=True та обчислювану властивість сумарної вартості товару total_value. Поле description відповідає міграційній схемі варіанта 3.

4.3. repositories.py

Реалізує патерн Repository для відокремлення запитів до БД від прикладної логіки.

```
from sqlalchemy import select

from sqlalchemy.orm import Session

from online_store.models import Category, Product


class CategoryRepository:

    def __init__(self, session: Session) -> None:

        self.session = session


    def add(self, name: str) -> Category:

        category = Category(name=name)

        self.session.add(category)

        self.session.commit()

        self.session.refresh(category)

        return category


    def get_by_id(self, category_id: int) -> Category | None:

        return self.session.get(Category, category_id)


    def get_by_name(self, name: str) -> Category | None:

        statement = select(Category).where(Category.name == name)

        return self.session.scalar(statement)
```

```
    def get_all(self) -> list[Category]:
        return list(self.session.scalars(select(Category)))

    def delete(self, category_id: int) -> bool:
        category = self.get_by_id(category_id)
        if category is None:
            return False
        self.session.delete(category)
        self.session.commit()
        return True

class ProductRepository:
    def __init__(self, session: Session) -> None:
        self.session = session

    def add(self, product: Product) -> Product:
        self.session.add(product)
        self.session.commit()
        self.session.refresh(product)
        return product

    def get_by_id(self, product_id: int) -> Product | None:
        return self.session.get(Product, product_id)

    def get_all(self) -> list[Product]:
        return list(self.session.scalars(select(Product)))
```

```
def find_by_name(self, name: str) -> list[Product]:
    statement = select(Product).where(Product.name.ilike(f"%{name}%"))
    return list(self.session.scalars(statement))

def find_by_category(self, category_id: int) -> list[Product]:
    statement = select(Product).where(Product.category_id == category_id)
    return list(self.session.scalars(statement))

def update_price_and_quantity(
    self, product_id: int, price: float | None = None, quantity: int | None = None
) -> Product | None:
    product = self.get_by_id(product_id)
    if product is None:
        return None
    if price is not None:
        product.price = price
    if quantity is not None:
        product.quantity = quantity
    self.session.commit()
    self.session.refresh(product)
    return product

def delete(self, product_id: int) -> bool:
    product = self.get_by_id(product_id)
    if product is None:
        return False
```

```
        self.session.delete(product)

        self.session.commit()

        return True
```

Інкапсулює CRUD-операції над сутностями через сесію SQLAlchemy. Забезпечує методи додавання, вибірки за ідентифікатором/атрибутами, оновлення характеристик та видалення записів без витоку деталей persistence layer у сервіси.

4.4. services.py

Містить аналітичні агрегації, транзакційну купівлю та роботу з DB-API.

```
import sqlite3

from sqlalchemy import func, select

from sqlalchemy.orm import Session

from online_store.models import Product


class StoreService:
    def __init__(self, session: Session) -> None:
        self.session = session


    def get_total_stock_value(self) -> float:
        """Обчислення загальної вартості залишків на складі (SUM(price * quantity))."""
        statement = select(func.sum(Product.price * Product.quantity))
        result = self.session.scalar(statement)
        return float(result) if result is not None else 0.0


    def get_most_expensive_product(self) -> Product | None:
        """Пошук найдорожчого товару."""
        statement = select(Product).order_by(Product.price.desc()).limit(1)
        return self.session.scalar(statement)


    def purchase_product(self, product_id: int, count_to_buy: int) -> Product:
        """
        Транзакція покупки товару:
        Перевіряє залишок, зменшує quantity та виконує commit.
```

```
        У разі нестачі залишків або збою виконується rollback (гарантія Atomicity).
        """
        if count_to_buy <= 0:
            raise ValueError("Кількість для покупки має бути більшою за нуль.")

        try:
            product = self.session.get(Product, product_id)
            if product is None:
                raise ValueError("Товар не знайдено.")

            if product.quantity < count_to_buy:
                raise ValueError(
                    f"Недостатньо товару на складі. Доступно: {product.quantity}, запитано:
{count_to_buy}"
                )

            product.quantity -= count_to_buy
            self.session.commit()
            self.session.refresh(product)
            return product
        except Exception:
            self.session.rollback()
            raise


def find_products_by_category_name_dbapi(
    database_path: str, category_name: str
) -> list[tuple]:
    """
    Параметризований пошук товарів через стандартний модуль sqlite3 (DB-API).
    Використовує '?' для запобігання SQL Injection.
    """
    with sqlite3.connect(database_path) as connection:
        cursor = connection.cursor()
```

```
    cursor.execute(
      """
      SELECT
        products.name,
        products.price,
        products.quantity,
        categories.name
      FROM products
      JOIN categories ON products.category_id = categories.id
      WHERE categories.name = ?
      ORDER BY products.price DESC
      """,
      (category_name,),
    )
    return cursor.fetchall()
```

Виконує агрегації через SQL-функції SUM та сортування ORDER BY. Метод purchase_product() реалізує бізнес-транзакцію з перевіркою залишку: у разі помилки викликається self.session.rollback(), що гарантує атомарність (ACID Atomicity). Функція find_products_by_category_name_dbapi() демонструє параметризований SQL-запит із використанням плейсхолдера ? для уникнення SQL-ін'єкцій.

4.5. main.py

Головна точка входу для демонстрації функціональності системи.

```
from online_store.database import DATABASE_FILE, SessionLocal, engine
from online_store.models import Base, Product
from online_store.repositories import CategoryRepository, ProductRepository
from online_store.services import StoreService, find_products_by_category_name_dbapi


def main() -> None:
  Base.metadata.create_all(engine)


  with SessionLocal() as session:
    cat_repo = CategoryRepository(session)
    prod_repo = ProductRepository(session)
    service = StoreService(session)
```

```
electronics = cat_repo.get_by_name("Електроніка")
if not electronics:
    electronics = cat_repo.add("Електроніка")

books = cat_repo.get_by_name("Книги")
if not books:
    books = cat_repo.add("Книги")

if not prod_repo.get_all():
    prod_repo.add(
        Product(
            name="Ноутбук Pro 15",
            price=45000.0,
            quantity=8,
            category_id=electronics.id,
            description="Потужний робочий ноутбук",
        )
    )
    prod_repo.add(
        Product(
            name="Бездротова миша",
            price=950.0,
            quantity=25,
            category_id=electronics.id,
            description="Ергономічна миша",
        )
    )
    prod_repo.add(
        Product(
            name="Чистий код",
            price=800.0,
```

```
            quantity=12,
            category_id=books.id,
            description="Класика для розробників",
        )
    )

    print("=== Список товарів ===")
    for p in prod_repo.get_all():
        print(f"[{p.id}] {p.name:20} | Ціна: {p.price:8.2f} грн | Кількість: {p.quantity}")

    print(f"\nЗагальна вартість залишків: {service.get_total_stock_value():.2f} грн")
    expensive = service.get_most_expensive_product()
    if expensive:
        print(f"Найдорожчий товар: {expensive.name} ({expensive.price} грн)")

    laptop = prod_repo.find_by_name("Ноутбук")[0]
    print(f"\nКупівля 2 шт. товару '{laptop.name}'...")
    service.purchase_product(laptop.id, 2)
    print(f"Залишок після покупки: {laptop.quantity} шт.")

  print("\n=== Демонстрація DB-API (товари категорії 'Електроніка') ===")
  dbapi_rows = find_products_by_category_name_dbapi(DATABASE_FILE, "Електроніка")
  for row in dbapi_rows:
      print(f"Назва: {row[0]:20} | Ціна: {row[1]:8.2f} | Кількість: {row[2]}")

if __name__ == "__main__":
  main()
```

Ініціалізує базу даних, наповнює її початковими записами, демонструє обчислення сумарної вартості складу, пошук найдорожчого товару, проведення транзакційної купівлі та прямий вибір даних через DB-API.

5. Файл pyproject.toml

Декларативна конфігурація збирання, метаданих та залежностей:

```
[build-system]
```

```
requires = ["setuptools>=70"]
build-backend = "setuptools.build_meta"

[project]
name = "online-store"
version = "0.7.0"
description = "Online Store persistence layer with SQLite and SQLAlchemy"
requires-python = ">=3.11"
dependencies = [
  "sqlalchemy>=2.0",
  "alembic>=1.13",
]

[project.optional-dependencies]
test = [
  "pytest>=8.0",
]

[project.scripts]
online-store = "online_store.main:main"
```

Визначає рушій збирання setuptools, версію Python 3.11+, основні залежності sqlalchemy, alembic, тестовий набір pytest та реєструє консольну команду запуску online-store.

6. Міграція бази даних (Alembic)

Файл ревізії migrations/versions/0001_add_description_to_products.py для розширення схеми:

```
"""add description to products

Revision ID: 0001
Revises:
Create Date: 2026-03-29
"""

import sqlalchemy as sa
from alembic import op
```

```
revision = "0001"
down_revision = None
branch_labels = None
depends_on = None


def upgrade() -> None:
  op.add_column(
    "products", sa.Column("description", sa.String(length=255), nullable=True)
  )


def downgrade() -> None:
  op.drop_column("products", "description")
```

Реалізує метод upgrade() для додавання стовпця description до таблиці products та зворотну дію downgrade() для видалення поля при відкаті міграції.

7. Файл .gitignore

Налаштування фільтрації тимчасових та автогенерованих файлів:

```
.venv/
__pycache__/
*.pyc
.pytest_cache/
*.db
*.egg-info/
dist/
build/
```

Захищає репозиторій від збереження локального оточення .venv, локальних файлів БД SQLite (*.db), байт-коду та збіркових артефактів.

8. Результат роботи програми

Фактичний результат виконання консольного модуля src/online_store/main.py:

```
=== Список товарів ===
[1] Ноутбук Pro 15      | Ціна: 45000.00 грн | Кількість: 8
[2] Бездротова миша     | Ціна:   950.00 грн | Кількість: 25
[3] Чистий код          | Ціна:   800.00 грн | Кількість: 12
```

Загальна вартість залишків: 393350.00 грн

Найдорожчий товар: Ноутбук Pro 15 (45000.0 грн)

Купівля 2 шт. товару 'Ноутбук Pro 15'...

Залишок після покупки: 6 шт.

=== Демонстрація DB-API (товари категорії 'Електроніка') ===

Назва: Ноутбук Pro 15 | Ціна: 45000.00 | Кількість: 6

Назва: Бездротова миша | Ціна: 950.00 | Кількість: 25

Вивід підтверджує коректну ініціалізацію таблиць, підрахунок аналітики, успішне списання кількості при транзакційній купівлі та отримання даних через прямий SQL-запит.

9. Результат виконання тестів (Integration Tests)

Тести виконуються на тимчасовій ізольованій базі даних у пам'яті (sqlite:///:memory:):

```
====================== test session starts ======================
platform win32 -- Python 3.13.0, pytest-9.2.0, pluggy-1.7.0 -- D:\Робочий стіл\проф. python\online_store\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: D:\Робочий стіл\проф. python\online_store
configfile: pyproject.toml
collected 7 items

tests/test_repositories.py::test_create_and_read_product PASSED [ 14%]
tests/test_repositories.py::test_update_product PASSED [ 28%]
tests/test_repositories.py::test_delete_product PASSED [ 42%]
tests/test_repositories.py::test_filter_by_category PASSED [ 57%]
tests/test_repositories.py::test_stock_value_and_most_expensive PASSED [ 71%]
tests/test_repositories.py::test_purchase_product_success PASSED [ 85%]
tests/test_repositories.py::test_purchase_product_insufficient_stock_rollback PASSED [100%]

====================== 7 passed in 0.37s ======================
```

Всі 7 інтеграційних тестів виконано успішно. Зокрема, тест test_purchase_product_insufficient_stock_rollback підтвердив коректність відкату транзакції (rollback) та збереження початкової кількості товару в разі виникнення помилки.

10. Робота з Git

У проєкті сформовано поетапну історію комітів для збереження архітектурних змін persistence layer.

Коміт 1: "Add SQLAlchemy database models"

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/online_store (main)

$ git add pyproject.toml .gitignore src/online_store/database.py src/online_store/models.py

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/online_store (main)

$ git commit -m "Add SQLAlchemy database models"

[main (root-commit) 4966f5a] Add SQLAlchemy database models

4 files changed, 93 insertions(+)

create mode 100644 .gitignore

create mode 100644 pyproject.toml

create mode 100644 src/online_store/database.py

create mode 100644 src/online_store/models.py

Ініціалізовано шар роботи з базою даних SQLite за допомогою SQLAlchemy 2.0. Створено файл pyproject.toml із залежностями проєкту, налаштовано виключення у .gitignore, додано конфігурацію сесій у database.py та реалізовано реляційні моделі Category і Product зі зв'язком One-to-Many у models.py.

Коміт 2: "Implement repository persistence layer and transactional operations"

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/online_store (main)

$ git add src/online_store/repositories.py src/online_store/services.py src/online_store/main.py

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/online_store (main)

$ git commit -m "Implement repository persistence layer and transactional operations"

[main 5888a0f] Implement repository persistence layer and transactional operations

3 files changed, 226 insertions(+)

create mode 100644 src/online_store/main.py

create mode 100644 src/online_store/repositories.py

create mode 100644 src/online_store/services.py

Реалізовано шар доступу до даних за патерном Repository (CategoryRepository, ProductRepository). У модулі services.py створено функції агрегації, параметризований запит через DB-API для фільтрації товарів за категорією та транзакційну купівлю purchase_product() із механізмом rollback при нестачі залишків. Модуль main.py забезпечує запуск та перевірку роботи програми.

Коміт 3: "Add database migrations"

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/online_store (main)

$ git add alembic.ini migrations/

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/online_store (main)

$ git commit -m "Add database migrations"

[main 0184d0a] Add database migrations

3 files changed, 78 insertions(+)

create mode 100644 alembic.ini

create mode 100644 migrations/env.py

create mode 100644 migrations/versions/0001_add_description_to_products.py

Налаштовано інструмент міграцій Alembic. Додано конфігураційний файл alembic.ini, підключено метадані моделей у migrations/env.py та створено ревізію 0001_add_description_to_products.py з операціями upgrade і downgrade для додавання поля опису товару.

10.1. Результат git status

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/online_store (main)

$ git status

On branch main

Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

Команда відображає актуальний стан робочого каталогу. Усі файли проєкту та пакета збережено й зафіксовано в комітах, незафіксовані зміни відсутні, а робоче дерево повністю чисте.

10.2. Результат git log –oneline

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/online_store (main)

$ git log --oneline

b82bf50 (**HEAD** -> **main**, **origin/main**) Add package init file

a7e7821 Add integration tests

0184d0a Add database migrations

5888a0f Implement repository persistence layer and transactional operations

4966f5a Add SQLAlchemy database models

Команда показує компактну історію комітів. Вивід демонструє виконання лабораторної роботи за логічними етапами: налаштування моделей БД (4966f5a), впровадження сховищ та бізнес-транзакцій (5888a0f), інтеграція міграцій Alembic (0184d0a), додавання автоматизованих тестів (a7e7821) та завершальна фіксація структури пакета (8e21a0f).

11. Висновок

У лабораторній роботі №7 створено persistence layer для інтернет-магазину на базі SQLite та SQLAlchemy ORM. Спроєктовано зв'язані таблиці categories та products (One-to-Many), а взаємодію з базою інкапсульовано за допомогою патерну Repository. Реалізовано CRUD-операції, SQL-агрегації та транзакцію купівлі purchase_product(), яка за принципами ACID виконує commit() при успішній оплаті або rollback() при нестачі товару. Через Python DB-API продемонстровано параметризований запит для захисту від SQL Injection. За допомогою Alembic успішно застосовано міграцію для додавання поля description, а коректність усіх операцій підтверджено 7 інтеграційними тестами у pytest на тестовій базі в пам'яті. У результаті отримано практичні навички роботи з реляційними БД, ORM, транзакціями та міграціями у Python.